

```

import numpy as np

import matplotlib.pyplot as plt

# -----

# Simulate ONE Brownian path on a fine grid, then "coarsen" it.

# This makes comparisons across different dt more meaningful because they

# come from the SAME underlying path.

# -----

```

```

def brownian_fine_path(T=1.0, dt_fine=1/1000, seed=0):

    """

    Simulate a standard Brownian motion B_t on [0, T] using a fine time step dt_fine.

    Returns:

        t_fine: time grid, shape (N+1,)

        B_fine: Brownian values, shape (N+1,)

    """

    rng = np.random.default_rng(seed)

    N = int(T / dt_fine)

    t_fine = np.linspace(0.0, T, N + 1)

```

```

    # Brownian increments: ΔB ~ Normal(0, dt_fine)

    dB = np.sqrt(dt_fine) * rng.standard_normal(N)

```

```

    # Build the path by cumulative sum, starting at B(0)=0

    B_fine = np.empty(N + 1)

    B_fine[0] = 0.0

    B_fine[1:] = np.cumsum(dB)

    return t_fine, B_fine

```

```

def coarsen_path(t_fine, B_fine, dt, dt_fine):

    """

    Coarsen the fine Brownian path to a coarser step dt by sub-sampling.

    Requires dt to be an integer multiple of dt_fine.

    Returns:

        t: coarse grid

        B: coarse values

    """

    m = int(round(dt / dt_fine))

    if not np.isclose(m * dt_fine, dt):

        raise ValueError("dt must be an integer multiple of dt_fine for coarsening.")

    t = t_fine[::m]

    B = B_fine[::m]

    return t, B

```

```

def polygonal_length(t, B):

```

```

"""

Compute the length of the polygonal curve (t, B(t)) in the plane:

```

```

    L = sum_k sqrt( (Δt)^2 + (ΔB)^2 )

```

```

"""

```

```

dt = np.diff(t)

```

```

dB = np.diff(B)

```

```

return np.sum(np.sqrt(dt**2 + dB**2))

```

```

def finite_difference_derivative_at(t, B, t0):

```

```

    """

```

```

    Central finite-difference approximation of derivative at a point t0:

```

```

        B'(t0) ≈ (B(t0+dt)-B(t0-dt)) / (2 dt)

```

```

    Assumes t0 is exactly on the grid and not at endpoints.

```

```

    """

```

```

    # Find index i such that t[i] = t0

```

```

    i = np.where(np.isclose(t, t0))[0]

```

```

    if len(i) == 0:

```

```

        raise ValueError("t0 is not on the grid. Choose t0 that lies on the grid.")

```

```

    i = int(i[0])

```

```

    if i == 0 or i == len(t) - 1:

```

```

        raise ValueError("t0 cannot be at the boundary for central difference.")

```

```

    dt = t[i] - t[i-1] # uniform step

```

```

    return (B[i+1] - B[i-1]) / (2 * dt)

```

```

# -----

```

```

# Main parameters

```

```

# -----

```

```

T = 1.0

```

```

dt_fine = 1/1000

```

```

seed = 42 # change seed if a different random sample path is needed

```

```

# Generate one fine Brownian path

```

```

t_fine, B_fine = brownian_fine_path(T=T, dt_fine=dt_fine, seed=seed)

```

```

# -----

```

```

# 6a: Plot Brownian motion for dt=1/10 and dt=1/100

```

```

# -----

```

```

for dt in [1/10, 1/100]:

```

```

    t, B = coarsen_path(t_fine, B_fine, dt=dt, dt_fine=dt_fine)

```

```

    plt.figure()

```

```

    plt.plot(t, B, marker='o', linewidth=1)

```

```

    plt.title(f"Brownian motion sample path (dt = {dt})")

```

```

    plt.xlabel("t")

```

```

    plt.ylabel("B(t)")

```

```
plt.grid(True)

plt.show()
```

```
# -----

# 6b: Show "infinite length" by plotting length vs dt as dt decreases

# We'll compute polygonal length for several dt values.

# -----

dt_list = [1/10, 1/20, 1/50, 1/100, 1/200, 1/500, 1/1000] # all divide dt_fine=1/1000

lengths = []
```

```
for dt in dt_list:

    t, B = coarsen_path(t_fine, B_fine, dt=dt, dt_fine=dt_fine)

    L = polygonal_length(t, B)

    lengths.append(L)
```

```
plt.figure()

plt.plot(dt_list, lengths, marker='o', linewidth=1)

plt.gca().set_xscale("log")

plt.title("Estimated path length vs discretization step dt")

plt.xlabel("dt (log scale)")

plt.ylabel("Polygonal length L(dt)")

plt.grid(True, which="both")

plt.show()
```

```
# -----

# 6c: Show non-differentiability by finite-difference derivative at a fixed t0

# Use central difference derivative approximation at t0=0.5 for many dt.

# -----

t0 = 0.5

derivs = []
```

```
for dt in dt_list:

    t, B = coarsen_path(t_fine, B_fine, dt=dt, dt_fine=dt_fine)

    d = finite_difference_derivative_at(t, B, t0=t0)

    derivs.append(abs(d)) # clearer for showing blow-up
```

```
plt.figure()

plt.plot(dt_list, derivs, marker='o', linewidth=1)

plt.gca().set_xscale("log")

plt.gca().set_yscale("log")

plt.title(f"|Finite-difference derivative at t0={t0}| vs dt")

plt.xlabel("dt (log scale)")

plt.ylabel("|approx derivative| (log scale)")

plt.grid(True, which="both")

plt.show()
```



